

CHAPTER 3

The Bottom of the Bowl

It was the autumn of 1959. Bernard Widrow, a young academic on the cusp of turning thirty, was in his office at Stanford University when a graduate student named Marcian “Ted” Hoff came looking for him. The young man arrived highly recommended. The day before, a senior professor at Stanford had reached out to Widrow on Hoff’s behalf, saying, “I’ve got this student named Ted Hoff. I can’t seem to get him interested [in my research]; maybe he’d be interested in what you’re doing. Would you be willing to talk with him?” Widrow replied, “Sure, happy to.”

“So, the next day, knocking on my door was Ted Hoff,” Widrow told me.

Widrow welcomed him in and proceeded to discuss his work, which was focused on adaptive filters—electronic devices that *learn* to separate signals from noise—and the use of calculus to optimize such filters. As Widrow chalked up the math on the blackboard, Hoff joined in, and soon the conversation morphed into something more dramatic. During that discussion, the two invented what came to be called the least mean squares (LMS) algorithm, which has turned out to be one of the most influential algorithms in machine learning, having proven foundational for those figuring out how to train artificial neural networks. “When I wrote the LMS algorithm on the blackboard for the first time, somehow I just knew intuitively that this is a

profound thing,” Widrow told me. “Too bad I didn’t have a camera to take a picture.”

Widrow grew up in a small town in Connecticut. He could hardly have imagined his luminous academic career. His father ran an ice-manufacturing plant. A curious young Widrow hung around the plant, amid generators, motors, and compressors, always asking questions. He admired the plant’s electrician, who taught him the basics of the trade. While Widrow was still in high school, his dad sat him down and asked, “What do you think you want to be when you grow up?”

The teen answered, “I want to be an electrician.”

His father said, “You don’t want to be an electrician. You want to be an electrical engineer.”

That subtle course correction led Widrow to MIT in 1947, where he obtained his bachelor’s, master’s, and doctorate; he joined MIT as an assistant professor in 1956. One day during the summer of that year, Widrow’s colleague Ken Shoulders came into the lab and told him about a workshop on artificial intelligence at Dartmouth College he was attending; did Widrow want to come? “I said, ‘What’s artificial intelligence?’ He said, ‘I don’t know. But it sounds interesting.’ So, I said, ‘Sure. I’ll go with you.’ ”

The coining of the term “artificial intelligence” is credited to John McCarthy, a mathematics professor at Dartmouth College. In August 1955, McCarthy; Marvin Minsky, who was then at Harvard University; Nathaniel Rochester of IBM; and Claude Shannon of Bell Telephone Laboratories, put out “A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence.” It began with a bold declaration:

We propose that a 2 month, 10 man study of artificial intelligence be carried out during the summer of 1956 at Dartmouth College in Hanover, New Hampshire. The study is to proceed on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it. An attempt will be made to find how to make machines use language, form abstractions and concepts, solve kinds of problems now reserved for humans, and improve themselves. We think that a significant advance can be made in one or more of these problems if a carefully selected group of scientists work on it together for a summer.

Widrow remembered it as an open seminar, no invitation needed. You could go and stay as long as you wanted. You spoke up if you had something to say, or you could simply listen. Widrow listened and then returned to MIT all charged up. He wanted to build a thinking machine. "I spent six months thinking about thinking," he said. "The conclusion was that with the circuitry, the technology that we had at that time, I expected it'd be twenty-five years before we'd be able to build a thinking machine." For a young researcher at the start of his career, it seemed a foolhardy venture. Widrow abandoned his plans and turned to something more concrete: adaptive filters that could learn to remove noise from signals. He was particularly interested in the digital form of adaptive analog filters developed by Norbert Wiener. (We encountered Wiener in the previous chapter as the man who coined the term "cybernetics.")

To understand Wiener's analog filter, consider some continuously varying (hence analog) signal source. Some noise is added to the signal, and the filter's job is to tell signal from noise. Wiener's filter theory showed how this could be done. Others adapted the theory to digital signals. Instead of being continuous, digital signals are discrete, meaning they have values only at certain points in time (say, once every millisecond). Widrow wanted to build a digital filter, but one that could learn and improve over time. In other words, it would learn from its mistakes and become a better version of itself.

At the heart of such an adaptive filter is a nifty bit of calculus. Imagine that the filter, at any given time, makes an error. Let's assume that we are able to keep track of such errors for ten time steps. We must reduce the error the filter makes by looking at its previous ten errors and adjusting its parameters. One measure of the mistakes is simply the average of the previous ten errors. However, errors can be positive or negative, and if you just add them to take the average, they can cancel each other out, giving the wrong impression that the filter's working well. To avoid this, take the square of each error (thus turning it into a positive quantity) and then take the average of the squares of the errors. There are other advantages to squaring the errors and averaging them that have to do with statistics and calculus, but we don't need to focus on those yet. The goal is to minimize this "mean squared error" (MSE) with respect to the parameters of the filter. To restate, we must change the values of the filter's parameters at each time step such that the average, or mean, of the squared errors of the past, say, ten steps is minimized. Understanding how this works requires delving into some simple calculus and learning a method that was

first proposed in 1847 by Baron Augustin-Louis Cauchy, a French mathematician, engineer, and physicist. It's called the method of steepest descent.

DOWN FROM ON HIGH



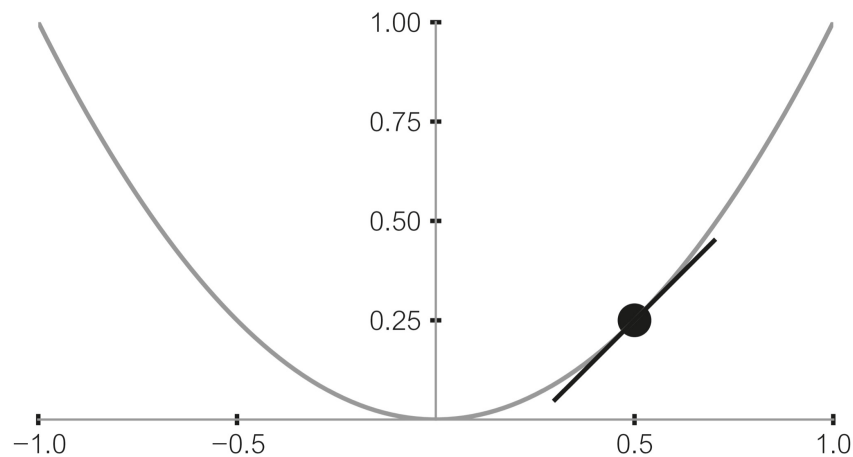
If you have seen pictures of—or, better yet, visited—rice paddies on hillsides, particularly in China, Japan, and Vietnam, you may have marveled at the flat terraces cut into the sides of the hills. If we walk along any one terrace, we remain at the same elevation. The edges of the terraces trace the contours of the terrain. Imagine standing on some terrace way up a hillside. Down in the valley below is a village. We have to get to the village, but it's getting dark, and we can see only a few feet ahead of us. Let's say the hillside is not too steep and that it's possible to clamber down even the steepest parts. How will we proceed?

We can stand at the edge of the terrace and look for the steepest route to the terrace below. That's also the shortest path down to the next piece of level ground. If we repeat

the process from terrace to terrace, we will eventually reach the village. In doing so, we will have taken the path of steepest descent. (This might not be a straight line from our initial position down to the village; we might have had to zigzag down the hillside.)

What we instinctively did was evaluate the slope, or the gradient, of the hillside as we looked in different directions while standing at the edge of a terrace and then took the steepest path down each time. We just did some calculus in our heads, so to speak.

More formally, let's look at going down a slope for a 2D curve, given by the equation $y = x^2$.



First, we plot the curve on the xy plane and then locate ourselves on the curve at some value of x , say, $x = 0.5$. At that point, the curve has a slope.

One way to find the slope of a curve is to draw a tangent to the curve at the point of interest. The tangent is a straight line. Imagine walking a smidgen along the straight line. You would be at a new location, where the x -coordinate has changed by an infinitesimal amount (Δx ; read that as “delta x ”) and the y -coordinate has also

changed by a corresponding infinitesimal amount (Δy). The slope is $\frac{\Delta y}{\Delta x}$. (If you think of climbing stairs, then the slope of the stairs is given by the rise divided by the run, where the rise is Δy , or how much you go up vertically with each step, and the run is Δx , the amount you move horizontally with each step.)

Of course, when you do this in our example, you have moved along the tangent to the curve, not along the curve itself. So, the slope really pertains to the tangent and not the curve. However, if the change in the x-direction, Δx , approaches zero, then the slope of the tangent line gets closer and closer to the slope of the curve at the point of interest, until the two become the same when $\Delta x = 0$. But how do you calculate the slope when the denominator in $\frac{\Delta y}{\Delta x}$ is zero? That's where calculus steps in.

Differential calculus is a branch of calculus that lets us calculate the slope of a continuous function (one that has no cusps, breaks, or discontinuities). It lets you analytically derive the slope in the limit $\Delta x \rightarrow 0$ (read that as "delta-x tends to zero"), meaning the step you take in the x-direction becomes vanishingly small, approaching zero. This slope is called the derivative of a function.

For our function $y = x^2$, the derivative equals $2x$. (Finding the derivative of a function is at the heart of differential calculus, but we won't get into the details here. For the functions used in this book, I'll simply provide the derivatives. To understand how to find the derivative of a function and for a list of derivatives of common functions, consult Wolfram MathWorld.)

We write our derivative as:

$$\frac{dy}{dx} = 2x$$

This is called the derivative of y with respect to x .

Calculus can cause our eyes to glaze over, if not induce downright dread. But as Silvanus P. Thompson, a professor of physics, electrical engineer, and member of the Royal Society, wrote in his classic *Calculus Made Easy* (first published in 1910), the “preliminary terror” of symbols in calculus “can be abolished once [and] for all by simply stating what is the meaning—in common-sense terms—of the...principal symbols.” The symbol d , he points out, simply means a “little bit of.” So, $\frac{dy}{dx}$ is a little bit of y divided by a little bit of x . The beauty of calculus is that you can calculate this ratio even as that “little bit of x ” tends to zero, or $dx \rightarrow 0$.

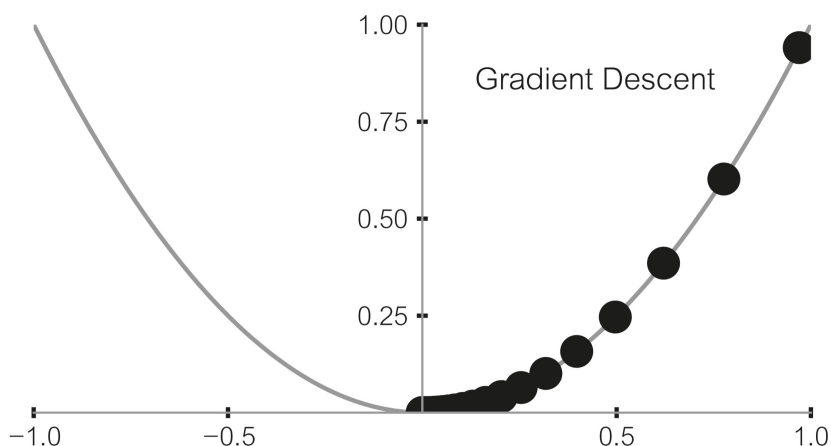
Given the derivative, you now have a way of determining the slope at any point along the curve. So, for the function $y = x^2$, the slope $\frac{dy}{dx} = 2x$, and at $x = 2$ the slope is equal to $2x$, which equals 4. At $x = 1$, the slope is 2; at $x = 0.5$, the slope is 1; and at $x = 0$, the slope is 0. You can see that when we move along the curve, as the value of x decreases from 2, so does the slope, until the function reaches a minimum (where the slope becomes zero), and then decreases further. The slope in general is zero at the minimum of a function; in this example, the (x, y) coordinates also happen to be $(0, 0)$ at the minimum, but that doesn’t have to be so.

We are now equipped to understand the method of steepest descent, also known as the method of gradient descent. Let's say we are at the coordinate (1, 1). We want to reach the bottom of the curve, where the slope is zero and the value of the function is at its minimum. At any point along the curve, there are only two ways you can go. Going one way takes you away from the bottom; going the other way brings you closer to the bottom. The trick to taking a step in the right direction is, first, to calculate the slope, or gradient, at your current location. (The term "gradient" has a more specific meaning, but let's use it here nonetheless.) In this case, for $x = 1$, the gradient is 2. If the function is bowl-shaped, as this one is, then the path toward the minimum involves going in a direction that decreases the gradient. So, we take a step such that the value of the x-coordinate is reduced by some step size (η) multiplied by the gradient at that point:

$$x_{new} = x_{old} - \eta \cdot \text{gradient}$$

$$y_{new} = x_{new}^2$$

Let's take a moment to see why taking a step that reduces the value of x reduces the gradient. For our equation, the gradient is given by $2x$. So, if the new value of x is less than the old value of x , the gradient at the new location will be lower than before. The new x-coordinate gives us a new y-coordinate. We end up at a new location. We repeat the process until the gradient becomes zero or close to zero. (We have reached the bottom or near enough to it.) Here's a graph depicting the process:

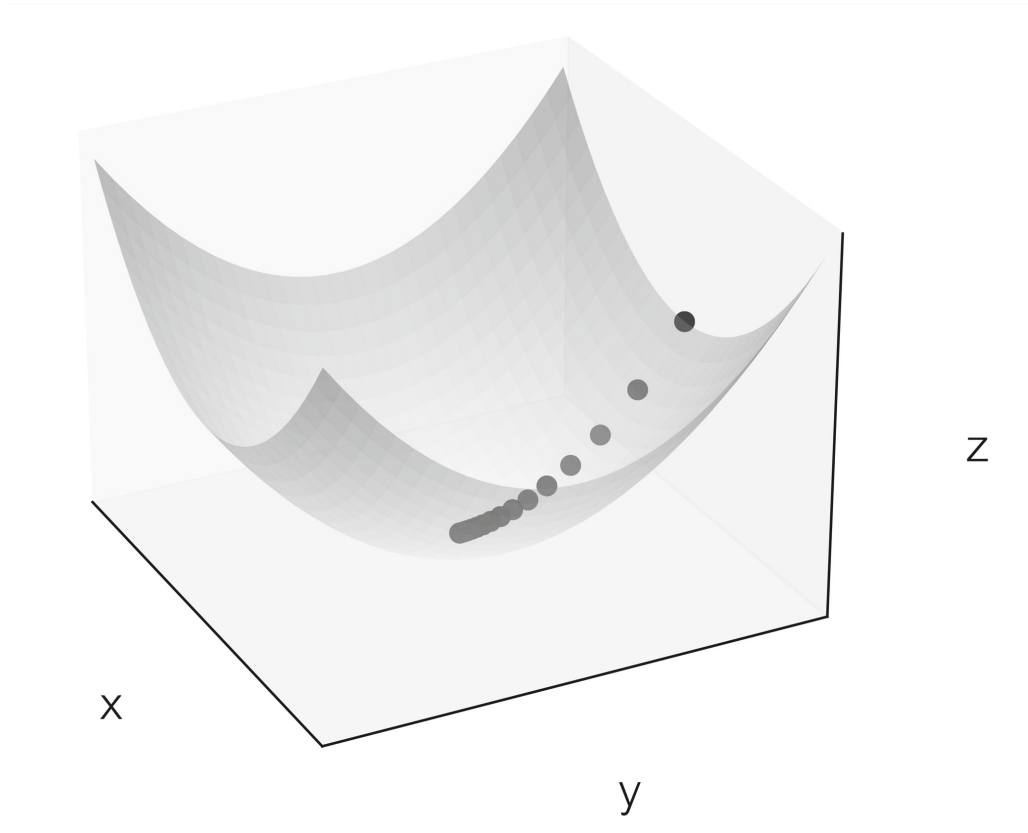


The step size, η , must be some small number, a fraction (say, 0.1). Why? Mainly because as you near the bottom, you want to be very careful you don't overshoot the minimum and end up higher on the other side of the curve. If you do so, depending on the function, the algorithm might begin taking you higher up the curve, away from the minimum. Also, notice that though the step size is the same at each iteration of the algorithm, the size of the jumps along the curve is greater in the beginning and becomes smaller as you near the bottom. Again, why? It's because we are subtracting a multiple of the gradient from the x-coordinate to get a new x-coordinate. The multiple, or step size, doesn't change in our algorithm. But what's changing is the gradient: It's getting smaller. So, the jumps along the curve also get progressively smaller.

Functions like the one depicted above, which have a single, well-defined minimum, are also called convex functions. When we find the bottom of the bowl, technically, we have found the "global" minimum of the function. (If a function has multiple minima, then each of these is called a "local" minimum.)

Now consider the case when the minimization involves a function that takes two inputs. Here's the function:

$$z = x^2 + y^2$$

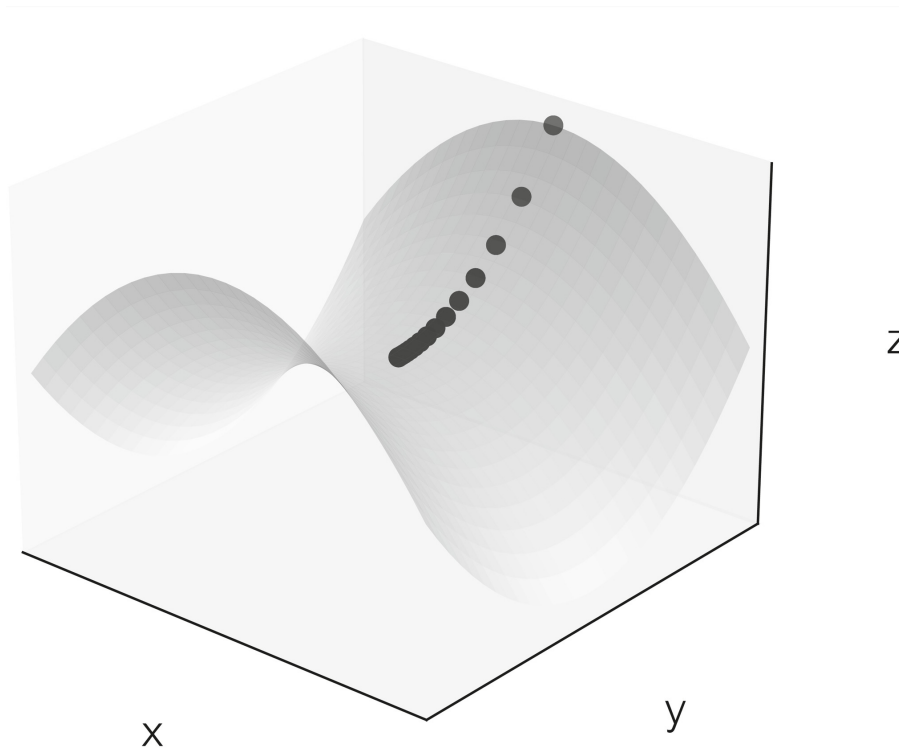


The plot shows the bowl-shaped 3D surface, called an elliptic paraboloid. If you start from any location above the bottom of the bowl, the descent along the surface of this paraboloid can be easily visualized. The difference from the 2D case is that we use the gradient at any location to calculate the new x and y coordinates, instead of just the x-coordinate. (Same operation: Subtract gradient times some step size from each coordinate.) This then gives us a new z-coordinate, and we come down to a new location on the surface. Doing this iteratively gets us to the bottom of the bowl.

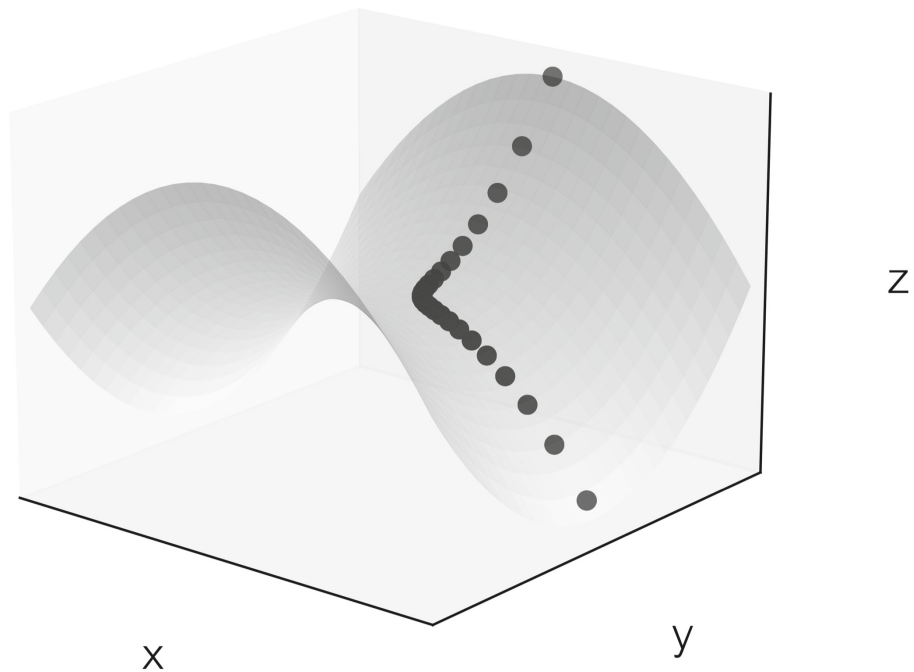
Here's a different function to help visualize why it may not be possible to find the minimum for certain functions.

The 3D surface is defined by this equation:

$$z = y^2 - x^2$$



The surface is called a hyperbolic paraboloid. Notice that it looks like a saddle: part convex surface and part concave. In the figure above, we start descending from our location and ostensibly reach a place where the gradient is zero. But this is an unstable place. It's called a saddle point. One false step, and you'll tumble down the surface. This function has no global or local minimum. Also, the initial starting point can dictate whether you even come close to the saddle point while descending. Take this scenario, for instance:



In this case, attempting to descend the gradient by following the same technique (because you start somewhere else) can cause you to veer away from the saddle point.

All this may sound terribly abstract, but gradient descent is crucial not only to Widrow and Hoff's algorithm, but also to modern machine learning. But before we connect gradient descent to Widrow and Hoff's work, there's one important detail we need to address.

Take this function again:

$$z = x^2 + y^2$$

Recall that when we had a function with one variable ($y = x^2$), we could use calculus to determine the derivative ($\frac{dy}{dx} = 2x$) and use this value to perform gradient descent.

But what do we do when the function involves multiple variables? Well, there's an entire field of so-called multi-variable, or multi-variate, calculus. And while it can be daunting to confront multi-variate calculus in its entirety, we can appreciate the central role it plays in machine learning by focusing on some simple ideas.

Imagine you are standing at some point on the surface of an elliptic paraboloid, $z = x^2 + y^2$. To figure out the direction of steepest descent, we must be concerned about two directions, given that we have two variables. Following Thompson's exhortation to state things in simple ways, we know that moving along the surface means a small change in the value of the variable z . So, our job is to calculate $\frac{\partial z}{\partial x}$ and $\frac{\partial z}{\partial y}$; or a "tiny change in z divided by a tiny change in x " and a "tiny change in z divided by a tiny change in y ," respectively.

In calculus-speak, we are taking the partial derivative of z with respect to x , and the partial derivative of z with respect to y . For our elliptic paraboloid, the partial derivatives are: $\frac{\partial z}{\partial x} = 2x$, $\frac{\partial z}{\partial y} = 2y$.

Also, note the slight change in the symbol used: ∂x instead of dx and ∂y instead of dy . The curvy "d" signifies a partial derivative of a function w.r.t. ("with respect to") one of many variables. For a conceptual understanding of what comes next, we don't need to worry about how to derive these partial derivatives. It's enough to know that given differentiable functions, calculus shows us how to get at these analytical expressions.

The most important concept here is that the direction of steepest descent, for this example, is given by two partial

derivatives. Let's say you are standing at a location where:

$$x = 3, y = 4, \text{ and } z = 3^2 + 4^2 = 25$$

At this location, the two partial derivatives have the values:

$$2x = 2 \times 3 = 6$$

$$2y = 2 \times 4 = 8$$

If you write these numbers in this form, it looks like something very familiar: $[6, 8]$. It's a vector!

So, if you have to move slightly in the direction of steepest descent, that direction can be inferred from this vector. Recall that a vector has a magnitude (or length) and a direction. In this case, our vector is an arrow going from $[0, 0]$ to $[6, 8]$. This vector is called the gradient. One technical point: The gradient points away from the minimum. So, to go down toward the minimum, you must take a small step in the opposite direction or follow the negative of the gradient.

If there's one thing to take away from this discussion, it's this: For a multi-dimensional or high-dimensional function (meaning, a function of many variables), the gradient is given by a vector. The components of the vector are partial derivatives of that function with respect to each of the variables.

For our elliptical paraboloid, the gradient is written as:

$$\begin{bmatrix} \partial z / \partial x \\ \partial z / \partial y \end{bmatrix} = \begin{bmatrix} 2x \\ 2y \end{bmatrix} \text{ or } \begin{bmatrix} 2x & 2y \end{bmatrix}$$

The gradient can be written as either a row vector or a column vector.

What we have just seen is extraordinarily powerful. If we know how to take the partial derivative of a function with respect to each of its variables, no matter how many variables or how complex the function, we can always express the gradient as a row vector or column vector. Just to illustrate the power of this approach, consider this slightly more complicated equation:

$$f(x, y, z) = x^2 + 3y^3 + z^5$$

The function f depends on three variables and is plotted in 4D space. There is no way for us to visualize what it looks like. And just by looking at the equation, it's impossible to tell whether the function has a global minimum toward which we can descend. But it's possible to write down the gradient using the partial derivatives. (Again, we are not trying to figure out how exactly to differentiate the function with respect to each variable; let's take it that if the function can be differentiated, calculus will provide an answer. You can use Wolfram MathWorld to find these derivatives.)

$$\begin{bmatrix} \partial f / \partial x \\ \partial f / \partial y \\ \partial f / \partial z \end{bmatrix} = \begin{bmatrix} 2x \\ 9y^2 \\ 5z^4 \end{bmatrix}$$

Now, given some set of values for x , y , and z , we can evaluate the gradient of the function at that point, take a small step in the opposite direction, and update the values

of x , y , and z . If the function has a global minimum or local minima, iterating over this process will get us there. Our analysis has also connected the dots between two important concepts: functions on the one hand and vectors on the other. Keep this in mind. These seemingly disparate fields of mathematics—vectors, matrices, linear algebra, calculus, probability and statistics, and optimization theory (we have yet to touch upon the latter two)—will all come together as we make sense of why machines learn.

GLIMMERS OF A NEURON

Bernard Widrow came back from the 1956 AI conference at Dartmouth with, as he put it, a monkey on his back: the desire to build a machine that could think. “It’s always there,” he told me more than six decades later. “I haven’t really ever gotten that out of my system.” Yet, in 1956, a young Widrow was savvy enough to realize the futility of building thinking machines, and he turned to more practical things. Building an adaptive filter was one such pursuit.

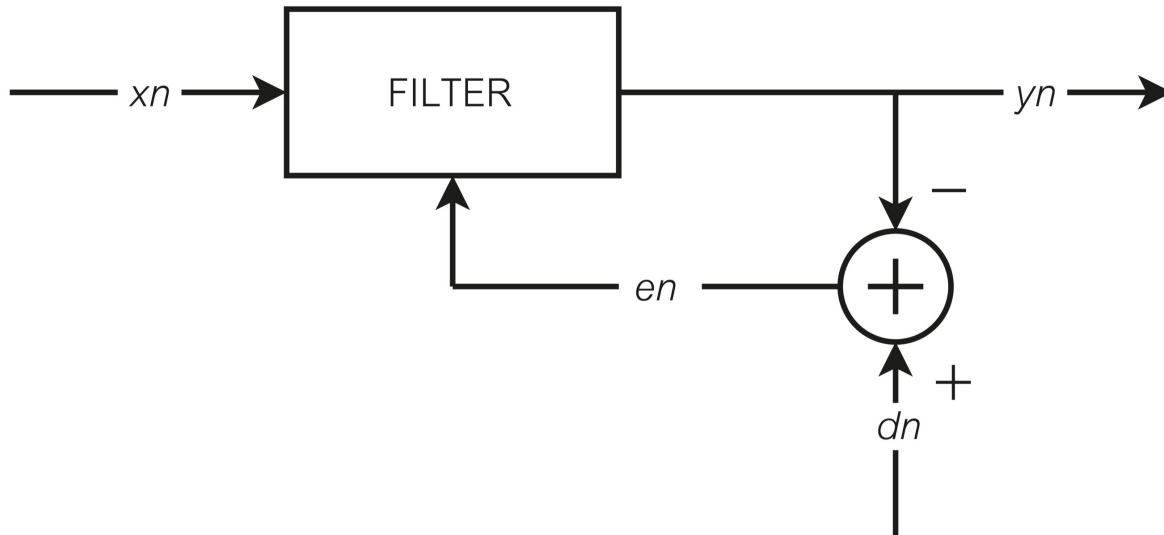
In the field of signal processing, a filter is something that takes an input signal, processes it, and produces an output signal that has certain desired properties. Let’s say you are working on some hobby electronics equipment, and you need to measure a signal. But mixed in with your signal is an annoying hum at a frequency of 60 Hz. That’s interference from the AC mains power. A filter can take the noise-laden input, remove only the 60 Hz component, and spit out a clean signal. Such a filter is easy to design, as the noise is well understood; it’s always at 60 Hz. But often, a filter needs to learn the characteristics of the noise; it needs to adapt.

Consider an important application for such an adaptive filter: digital communications. Anyone who has ever used a dial-up modem to connect to the internet will remember the distinctive sounds made by the modem. First a dial tone, then the tones of the number being dialed, followed by beeps and bursts of staccato screeches, and then silence after about twenty seconds. That's the sound of a handshake: two digital devices figuring out the best way to talk to each other over a phone line usually used for analog voice signals. The digital devices must transmit and receive streams of zeroes and ones. But analog transmission lines can be noisy—so, you need a filter to remove the noise that could corrupt the data. This includes canceling any echo that a modem may hear of its own transmissions. But it's impossible to build a generic filter for such purposes. The noise can be, and often is, different in each instance of two communicating devices. Part of what happens during a handshake is that an adaptive filter at each end figures out the characteristics of the noise, which it can then remove to create an almost error-free communication channel. (Widrow recalls using a fax machine that made these "handshake" sounds when communicating with some remote fax machine; his grandson, who happened to be around him in those days, started calling the sounds of the handshake "Grandpa music.")

One design of an adaptive filter is shown below.

Here, x_n is the input signal; y_n represents the corresponding output. The filter turns x_n into y_n . The output is compared against a desired signal d_n , which is the signal the filter ought to have produced. Any discrepancy between y_n and d_n results in an error e_n .

$$e_n = d_n - y_n$$



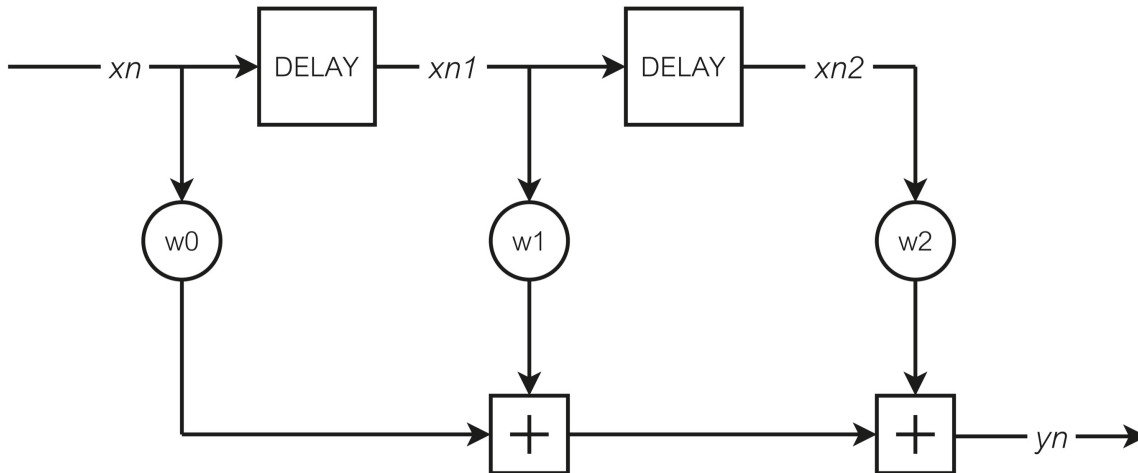
This error e_n is fed back into the filter. An adaptive filter changes itself such that the error is minimized. The black box named FILTER has some characteristics, or parameters, and these parameters can be tweaked to make the filter adaptive.

You might say, if you know the desired signal, what's the point of a filter? Well, you don't know the desired signal for any generic input. But there are ways to know what the filter should produce for known inputs. For example, that's what modems do during the handshake: They transmit a previously agreed-upon signal, so that the other side knows what to expect. That's the desired signal d_n . But the signal arrives over a noisy transmission line, so the input x_n is simply d_n contaminated by noise. But unlike the 60 Hz hum we looked at earlier, this noise is random. The receiver needs a filter that takes x_n as an input and produces a signal y_n that is as close to the desired signal d_n as possible. To do so, the algorithm must learn the statistical properties of the noise, so that it can predict the noise at each time step and subtract it in real time from x_n to produce the desired signal.

While all this is a far cry from AI and ML, we can see glimmers of machines that learn. This connection—particularly to Rosenblatt’s perceptron and artificial neurons—will become even more obvious when we write down the particulars of a filter.

This was slowly becoming obvious to Widrow, too, while he was still at MIT, where he was deeply influenced by the doyen of filter design, Norbert Wiener. At the time, Wiener was MIT’s best-known professor. Decades later, Widrow, recalling Wiener’s personality in a book, painted a particularly evocative picture of a man whose head was often, literally and metaphorically, “in the clouds” as he walked the corridors of MIT buildings: “We’d see him there every day, and he always had a cigar. He’d be walking down the hallway, puffing on the cigar, and the cigar was at angle θ —45 degrees above the ground. And he never looked where he was walking...But he’d be puffing away, his head encompassed in a cloud of smoke, and he was just in oblivion. Of course, he was deriving equations.” Even as he approached the steps at the end of some hallway, Wiener would be looking up, not down. “You can see he’s going to kill himself—he’s going to fall down those steps—but if you disturb him, you might break his train of thought and set science back like ten years! There was always that problem.”

Such life-and-death decisions notwithstanding, Widrow embraced Wiener’s work. While at MIT, he even came up with different versions of the adaptive filter. Here’s an example of one of his designs:



In the filter, the input signal x_n arrives discretely once every n th time step (which can be anything: once per day, per second, per millisecond, per microsecond, and so on), and y_n is the corresponding output. Each box labeled DELAY takes a signal and delays it by one time step, producing the signal x_{n1} from x_n and x_{n2} from x_{n1} . After one delay, the signal is multiplied by a weight w_1 , after two delays it is multiplied by w_2 , and so on. The undelayed signal is multiplied by w_0 . All these are summed up. So, for our example on the preceding page, the output signal y_n can be written as:

$$y_n = w_0.x_n + w_1.x_{n1} + w_2.x_{n2}$$

We can treat $[w_0, w_1, w_2]$ as the vector $\mathbf{w}$, and $[x_n, x_{n1}, x_{n2}]$ as the vector $\mathbf{x}_n$. Then,

$$\mathbf{w}.\mathbf{x}_n = w_0.x_n + w_1.x_{n1} + w_2.x_{n2}$$

The diagram shows only two delays, but in principle, there can be any number of them. Now, if d_n is the desired signal, here's how you would go about optimizing the

filter's parameters, to minimize the error between what it generates, which is yn , and the desired signal, dn .

$$yn = \mathbf{w} \cdot \mathbf{xn}, \text{ where:}$$

$$\mathbf{xn} = [xn, xn1, \dots]$$

$$\text{and } \mathbf{w} = [w0, w1, \dots]$$

$$en = dn - yn$$

$$\Rightarrow en = dn - \mathbf{w} \cdot \mathbf{xn}$$

What we have is an expression for the error that the filter makes at the n th time step. It's clear that if the filter predicts a good approximation of the desired signal, then the error will be minimized. To achieve this, the filter must learn the value for $\mathbf{w}$ at each time step. Of course, such a filter can update its parameters whenever it gets the prediction wrong—hence the name “adaptive filter.” It learns. Ideally, over time, the average error made by the filter should tend toward zero. (Maybe now the connections to machine learning are beginning to emerge from this fog of filter theory.)

A small ML digression: How should we calculate the average error? Adding the errors to calculate the mean won't suffice; as we saw earlier, negative and positive errors can negate each other, giving an invalid impression that the average error is low. We could add the absolute value of the errors and take the average: this is called the mean absolute error (MAE). But the math folk prefer to take the average of the square of the error terms and call it

a mean squared error (MSE). It turns out that the MSE has some nice statistical properties that the MAE doesn't. Also, the MSE is differentiable everywhere. (A differentiable function is one that has a derivative everywhere in its domain, where a domain could be, say, the xy plane.) The MAE is not. This, too, helps immensely, and we'll see this when we come to training neural networks. One more fact worth mentioning: If you want your error estimate to punish the extreme outliers, then the MSE does that better than the MAE, because the contribution of an error to the average increases as the square of the error in the MSE, while it increases linearly in the MAE.

Back to the filter. We square the error at each time step, add all the squared errors, and then find the expected value. The expected value, or "expectation," of something that is varying randomly has a very specific meaning in probability theory, but let's not worry about that. The key insight here is that we need to minimize the expected value (E) of the squared errors. Let's call that value J :

$$J = E(en^2)$$

$$\Rightarrow J = E((dn - yn)^2)$$

$$\Rightarrow J = E((dn - \mathbf{w} \cdot \mathbf{x}n)^2)$$

The value of J must be minimized. If you look at the form of the equation that relates J to the filter parameter $\mathbf{w}$, it becomes clear that the function that connects the two will be quadratic (meaning, it'll involve the second power of $\mathbf{w}$). We already saw that such quadratic functions are convex ($y = x^2$, or $z = x^2 + y^2$, for example). So, when J is minimized,

we end up at the bottom of some bowl-shaped function. At this point, the slope, or gradient, of J is zero. This gives us another way of finding the optimal value for $\mathbf{w}$. We can simply set the value of the gradient of J with respect to $\mathbf{w}$ to zero and solve the equation:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{\partial E\left((dn - \mathbf{w} \cdot \mathbf{x}_n)^2\right)}{\partial \mathbf{w}} = 0$$

In 1931, Wiener and German mathematician Eberhard Hopf devised a way to solve such equations, using techniques from linear algebra. But this requires some a priori knowledge about the correlation between the inputs at all the various time steps and the correlation between the inputs and the desired outputs. This is not always known, and even when it is, the calculations can be computationally intensive. Also, Wiener's work applied to analog filters.

We can also minimize J by using the method of steepest descent. Why is that? Well, because it's a bowl-shaped, convex function, we can always find the value for $\mathbf{w}$ that minimizes the expectation value of the squared errors by iteratively following a path down to the bottom of the bowl. So, regardless of whether the filter is characterized by one coefficient (w_0), two (w_0, w_1), three (w_0, w_1, w_2), or more, the assertion holds. Steepest descent will let you find the minimum. But this method, too, has a limitation: We need to be able to calculate the partial derivatives of J with respect to the filter coefficients.

There are other computational concerns.

For instance, given $\mathbf{x}_n$ and the corresponding y_n , plus the desired output dn , one can use the method of steepest descent to calculate the parameters (in our example: w_0 ,

$w1$, $w2$). The trouble is that to find the optimal values for the parameters, you need more and more samples of input, output, and desired output, and these calculations take increasingly longer to finish.

In addition, given that the error calculated for a certain sample of data doesn't fully represent all possible errors, the gradient you calculate at each time step to go toward the minimum is only an approximation. Sometimes it's pointing in the right direction, but most times it's not. In our analogy of walking down a terraced hillside to the village, it's as if in addition to having to navigate in the dark, you were somewhat drunk. Instead of taking the steepest path down to the next terrace, you stagger down willy-nilly. You might even clamber up to the next terrace. The hope is that if you take small enough steps, even this drunkard's walk will get you down to the village. And in practice, algorithms that do this indeed succeed. This method is called stochastic gradient descent (SGD), where the word "stochastic" refers to the fact that the direction of each step in your descent is slightly random.

This is what Widrow was working on while at MIT, before he moved to Stanford. But alongside filters, he was thinking about adaptive neurons, or neural elements, and realizing that training a neuron was no different from training a filter.

A WEEKEND WITH BERNIE

When Ted Hoff walked into Widrow's office at Stanford that fateful day in the fall of 1959, Widrow began discussing such ideas with him. "So, I was up at the blackboard explaining to Ted about the stochastic gradient and quadratic bowl...and adaptive filters and adaptive neural

elements and...talking about how you differentiate to get the components of the gradient,” Widrow told me. “I don’t know how it happened, but we came up with an idea that we could get a stochastic gradient, a very crude gradient algebraically—without differentiating anything, without averaging anything, and without squaring anything.”

The technique they devised could be applied to adaptive filters or artificial neurons. So far, we have learned that the output of our example adaptive filter is given by:

$$y_n = w_0.x_n + w_1.x_{n1} + w_2.x_{n2}$$

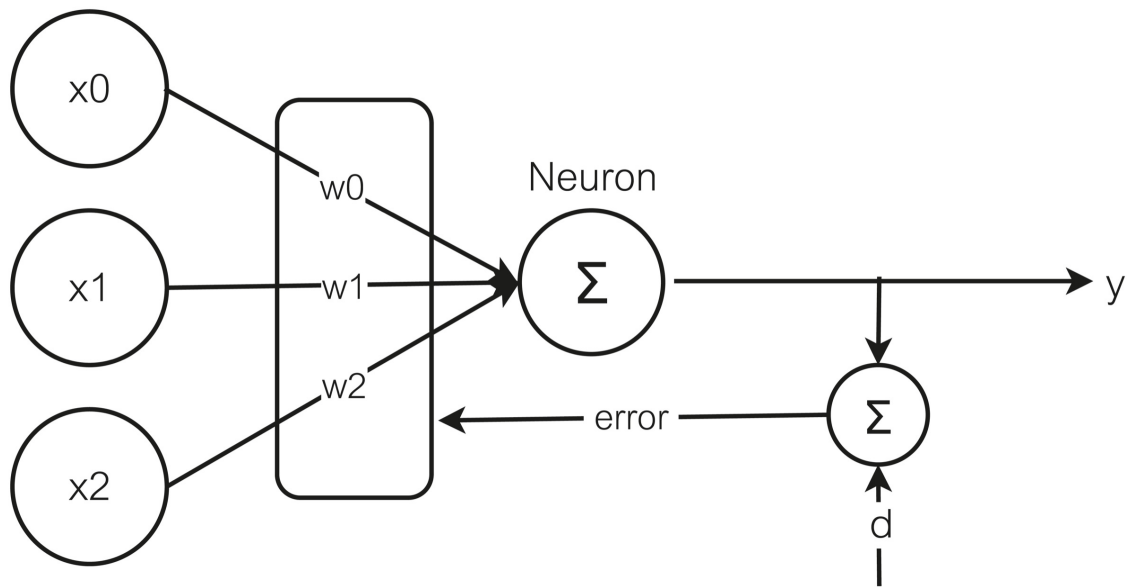
$$\text{or, } y_n = \mathbf{w}.\mathbf{x}_n$$

Designing a filter that adapts involves learning the values for w_0 , w_1 , and w_2 . If you cast your mind back to Rosenblatt’s perceptron, you’ll see that it, too, involves learning the weights so that it can correctly classify a new piece of data as falling on one or the other side of a hyperplane. Rosenblatt’s algorithm isn’t cast in terms of gradient descent. But the Widrow and Hoff algorithm is.

The figure below shows a way to think about the adaptive neuron designed by Widrow and Hoff.

The neuron produces an output y :

$$y = w_0x_0 + w_1x_1 + w_2x_2$$



Here, x_0 is always 1; this makes w_0 our bias term, b . The actual inputs are x_1 and x_2 . Together, they make up the vector $\mathbf{x}$. The set of coefficients w_0 , w_1 , and w_2 is the vector $\mathbf{w}$. So:

$$y = \mathbf{w} \cdot \mathbf{x}$$

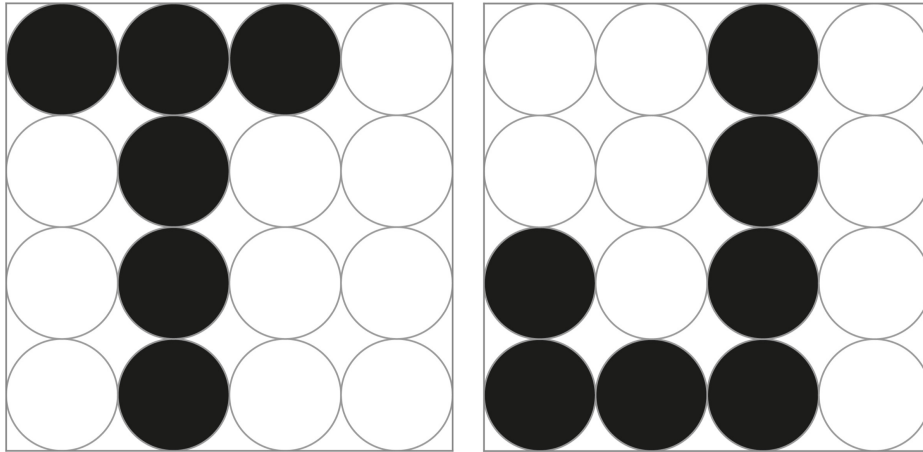
$$\Rightarrow y = \mathbf{w}^T \mathbf{x}$$

Assume you have several training samples for which you have the inputs and the corresponding desired outputs (d). Then, the error made by the adaptive neuron for each input is given by:

$$error(e) = d - y = d - \mathbf{w}^T \mathbf{x}$$

Consider the problem where the input is a set of 16 values, representing a 4×4 grid of pixels. These pixels can be used to show letters of the alphabet. The letter "T," for

example, would light up some of those pixels (meaning, some pixels would have the value “1,” and others “0”). The letter “J” would light up a different set of pixels.



Let’s say that when the set of pixels representing “T” is the input to the neuron, it must output the value 1. And when the input is the set of pixel values representing the letter “J,” the neuron must output -1. So, the desired output for “T” is 1, and for “J,” it’s -1.

Training the neuron involves supplying it one input, representing one letter, at a time. The algorithm uses the input and the desired output to adjust its weights and generate the correct output. But changing the weights so that you get the correct output for the input letter “T” may cause it to make an error for the input letter “J.” If that’s the case, the algorithm adjusts its weights again. Of course, now the new weights may cause an error for the input letter “T.” You repeat the process. And this goes on until the neuron correctly outputs 1 for the letter “T” *and* -1 for the letter “J.” The method of steepest descent can be used to train the neuron.

Let’s say you had a bunch of training samples: inputs and their corresponding outputs. If you calculate the errors

made by the neuron for all input samples and plot the expectation value of squared errors as a function of all the weights, or coefficients, you get a bowl-shaped function (of course, in a higher-dimensional space that we cannot visualize). Then, you could minimize the expectation value using the method of steepest descent. At each step, you calculate the gradient of the function with respect to each weight and then modify the weights by taking a small step in the opposite direction (toward the minimum).

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \mu(-\Delta)$$

where:

μ = step size

Δ = gradient

Recall from our previous discussion that the gradient is simply a vector in which each element is the partial derivative of the mean squared error, J , with respect to each weight.

So, for our three weights, the gradient is:

$$\left[\begin{array}{ccc} \frac{\partial J}{\partial w_0} & \frac{\partial J}{\partial w_1} & \frac{\partial J}{\partial w_2} \end{array} \right]$$

Each element of this vector will be an analytic expression that can be calculated using the rules of calculus. Once you have the expressions, you just plug in the current values for the weights, and you get the gradient, which you can then use to calculate the new

weights. The problem: You need calculus, and while our gradient has only three elements, in practice, it can have elements that number in the tens, hundreds, thousands, or even more. Widrow and Hoff were after something simpler. This is what they came up with:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \mu(-\Delta_{\text{est}})$$

Instead of calculating the entire gradient, they decided to calculate only an estimate of it. The estimate would be based on just one data point. It didn't involve calculating the expectation value of the error squared. Rather, they were simply estimating it. But estimating a statistical parameter based on just one sample is usually anathema. Even so, Widrow and Hoff went with it. With a little bit of analysis, they came up with their update rule for the weights:

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + 2\mu\epsilon\mathbf{x}$$

where:

μ = step size

ϵ = error based on one data point

$\mathbf{x}$ = the vector representing a single data point

The error itself is given by:

$$\epsilon = d - \mathbf{w}^T\mathbf{x}$$

This is simple algebra. Basically, for each input, you calculate the error and use that to update the weights.

Widrow and Hoff were aware that their method was extremely approximate. “What you do is you take the single value of the error, square it, swallow hard, because you are going to tell a lie, [and] you say that’s the mean squared error,” Widrow told me. “It’s a pretty noisy version of the average of the square of the error. And then, when you take the derivatives, you can do it analytically, without differentiating. You don’t have to square anything. You don’t have to average anything. You got an extremely noisy gradient. You take a small step, another small step, another small step.”

And yet, the algorithm gets you close to the minimum of the function. It came to be called the least mean squares (LMS) algorithm. In a video Widrow uploaded in 2012, to explain the algorithm, he credited one of his graduate students for naming the algorithm, but he doesn’t remember the student’s name. He also said, “I hope that all this algebra didn’t create too much mystery. It’s all quite simple once you get used to it. But unless you see the algebra, you would never believe that these algorithms could actually work. Funny thing is they do. The LMS algorithm is used in adaptive filters. These are digital filters that are trainable...Every modem in the world uses some form of the LMS algorithm. So, this is the most widely used adaptive algorithm on the planet.”

Not only would the LMS algorithm find uses in signal processing, but it would also become the first algorithm for training an artificial neuron that used an approximation of the method of steepest descent. To put this into context: Every deep neural network today—with millions, billions, possibly trillions of weights—uses some form of gradient

descent for training. It would be a long road from the LMS algorithm to the modern algorithms that power AI, but Widrow and Hoff had laid one of the first paving stones.

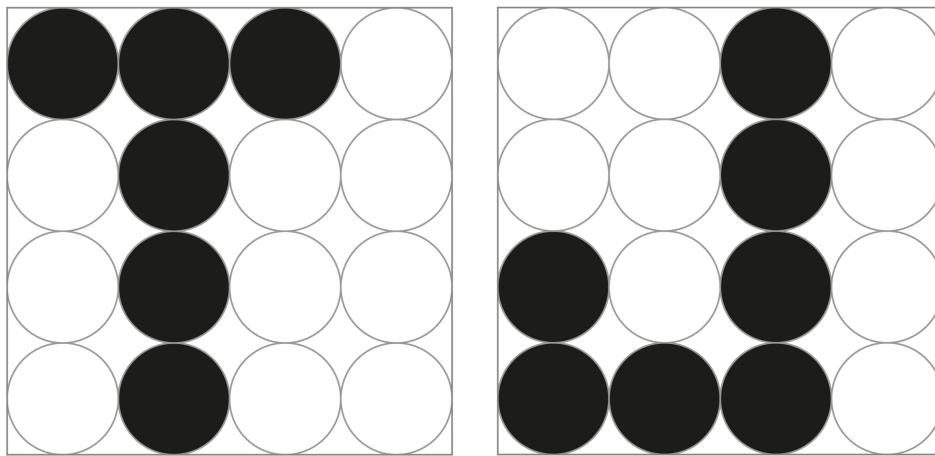
On that Friday afternoon in the autumn of 1959, however, all they had were mathematically motivated scribbles on a blackboard. Widrow and Hoff didn't know that the algorithm would work. They needed to simulate it on a computer; they were excited that they had uncovered something extremely important. "Foolishly I was thinking: 'We've discovered the secret of life,' " Widrow told me.

Across the hall from his office was an analog computer, a gift to Stanford from Lockheed. The door was open, and anyone could use the computer. Programming it was akin to operating an old-fashioned telephone switchboard: Take a wire out from a patch panel here, plug it in there, and so on. In a half hour, Hoff had the algorithm running on the analog machine. "He made it work," Widrow said. "I don't know how he knew how to do it. He knew how to program that thing."

Having verified that the algorithm worked, the two had as their next step the building of a single adaptive neuron—an actual hardware neuron. But it was late afternoon. The Stanford supply room was closed for the weekend. "Well, we weren't going to wait," Widrow told me. The next morning, the two of them walked over to Zack Electronics, in downtown Palo Alto, and bought all the parts they needed. They then went over to Hoff's apartment and worked all of Saturday and most of Sunday morning. By Sunday afternoon, they had it working. "Monday morning, I had it sitting on my desk," Widrow recalled. "I could invite people in and show them a machine that learns. We called it ADALINE—'adaptive linear neuron.' It was...not an

adaptive filter, but an adaptive neuron that learned to be a good neuron.”

What ADALINE does, using the LMS algorithm, is to separate an input space (say, the 16-dimensional space defined by 4×4 , or 16, pixels) into two regions. In one region are 16-dimensional vectors, or points that represent, say, the letter “T.” In another region are vectors that represent the letter “J.” Widrow and Hoff chose 4×4 pixels to represent letters, as this was big enough to clearly show different letters, but small enough to work with, given that they had to adjust the weights by hand (using knobs). Anything larger, and they’d have spent most of their time twiddling those knobs. Again, here are the letters “T” and “J” in 4×4 -pixel space:



So, each letter is represented by 16 binary digits, each of which can be either 0 or 1. If you were to imagine plotting these letters as points in a 16D space, then “J” would be a point (vector) in one part of the coordinate space, and “T” in another. The LMS algorithm helps ADALINE find the weights that represent the linearly separating hyperplane—in this case, a plane in fifteen

dimensions—that divides the input space into two. It's exactly what Rosenblatt's perceptron does, using a different algorithm.

While the perceptron convergence proof we saw in chapter 2 showed clearly why the perceptron finds the linearly separating hyperplane, if one exists, it wasn't exactly clear why the rough-and-ready LMS algorithm worked. Years later, Widrow was waiting for a flight in Newark, New Jersey. He had a United Airlines ticket. "Those days, your ticket was in a jacket. And there was some blank space on it. So, I sat down and started doing some algebra and said, 'Goddamn, this thing is an unbiased estimate.' "

He was able to show that the LMS algorithm, if you took extremely small steps, got you to the answer: the optimal value for the weights of either the neuron or the adaptive filter. "By making the steps small, having a lot of them, we are getting an averaging effect that takes you down to the bottom of the bowl," Widrow said.

Hoff finished his Ph.D. with Widrow and was doing his postdoctoral studies when a small Silicon Valley start-up came calling. Widrow told him to take the job. It was sound advice: The start-up was Intel. Hoff went on to become one of the key people behind the development of the company's first general-purpose microprocessor, the Intel 4004.

Widrow continued using the LMS algorithm to build adaptive filters—for noise cancelation and for antennas that adapt to remove noise and interference. He worked on ADALINE (a single layer of adaptive neurons) and MADALINE (for "Many ADALINE"), which had three layers: input, hidden, and output. But it was hard to train MADALINE. Still, Widrow's work began making waves.

In a 1963 episode of *Science in Action* titled “Computers that Learn,” produced by the California Academy of Sciences, the host and presenter, Earl S. Herald, introduces a robotic assembly that seems to be balancing a broom. “This may not seem very startling, because anyone can balance a broom. But this is a machine that can learn to do this...This is the story of MADALINE, the machine that in some respects *thinks like a man* [italics mine].” The sexism of the time rears its head again when Herald, barely two minutes into the episode, queries Widrow: “Let me ask you about the name ‘ADALINE.’ Why ‘ADALINE’? Why not a masculine name?” Widrow replies, “Well, this happens to spell ‘Adaptive Linear Neuron.’ And that’s it.”

The line connecting ADALINE to modern neural networks (which have multiple layers and are trained using an algorithm called backpropagation) is clear. “The LMS algorithm is the foundation of backprop. And backprop is the foundation of AI,” Widrow told me. “In other words, if you trace it back, this whole field of AI right now, [it] all starts with ADALINE.”

In terms of the backpropagation algorithm, this is a fair assessment. Of course, Rosenblatt’s perceptron algorithm can make similar claims. Together, Rosenblatt and Widrow laid some of the foundation stones for modern-day deep neural networks. But these weren’t the only such efforts. Other algorithms, also foundational, were being invented, and they’d rule the roost in the decades during which neural network research floundered, primarily because of Minsky and Papert’s unduly harsh assessment of its limitations. And these non-neural network approaches were establishing the governing principles for machines that learn based on, for example, probability and statistics, our next waystation.

CHAPTER 4

In All Probability

Probability deals with reasoning in the presence of uncertainty. And it's a fraught business for the best of us. There's no better illustration of how uncertainty messes with our minds than the Monty Hall dilemma. The problem, named after the host of the American television show *Let's Make a Deal*, became a public obsession in 1990 when a reader of the *Parade* magazine column "Ask Marilyn" posed the following question to columnist Marilyn vos Savant:

"Suppose you are on a game show, and you're given the choice of three doors. Behind one is a car; behind the others, goats. You pick a door, say, No. 1, and the host, who knows what's behind the doors, opens another door, say, No. 3, which has a goat. He then says to you, 'Do you want to pick No. 2?' Is it to your advantage to switch your choice?" The person playing the game has a quandary. Do they switch their choice from door No. 1 to door No. 2? Is there any benefit to doing so, in that they will increase their odds of choosing the door hiding the car? Before we look at vos Savant's answer, let's try to tackle the problem ourselves. Here's my intuitive answer:

Before the host opens one of the doors, the probability that a car is behind the door I've picked (Door No. 1) is one-third. But then the host opens Door No. 3 and reveals that there's a goat behind it. Now there are two closed doors, and behind one of them is the car. I figure that the